The Visual Computer
https://doi.org/10.1007/s00371-024-03385-4

RESEARCH

Generating natural pedestrian crowds by learning real crowd trajectories through a transformer-based GAN

Dapeng Yan[1] · Gangyi Ding[1] · Kexiang Huang[1] · Tianyu Huang[1]

Accepted: 24 March 2024

Abstract
Traditional methods for constructing crowd simulations often have shortcomings in terms of realism, and data-driven methods are an effective approach to enhancing the visual realism of crowd simulation. However, existing work mainly constructs crowd simulations through prediction-based approaches based on deep learning or by fitting the parameters of traditional methods, which limits the expressiveness of the model. In response to these limitations, this paper introduces a method capable of generating realistic pedestrian crowds. This approach uses a Generative Adversarial Network, complemented by a transformer module, to learn behavioral patterns from actual crowd trajectories. We use a transformer module to extract trajectory features of the crowd, then convert the spatial relationships between individuals into sequences using a special data processing mechanism, and use the transformer module to extract social features of the crowd, while guiding the movement of each individual with their target direction. During training, we simultaneously learn from real crowd data and simulation data resolving collisions by traditional methods, to enhance the collision avoidance behavior of virtual crowds while maintaining the movement patterns of real crowds, resulting in more general collision avoidance behavior. The crowds generated by the model are not limited to specific scenarios and show generalization capabilities. Compared to other models, our method shows better performance on publicly available large-scale pedestrian datasets after training. Our code is publicly available at https://github.com/ydp91/NPCGAN.

Keywords Crowd simulation · GAN · Pedestrian motion · Transformer

1 Introduction

Crowd simulation, a vital topic in computer graphics, finds diverse applications in game design [1–3], film production [4], military training [5], and urban science research [6]. Simulating virtual crowds can also have significant implications for the real world. However, most crowd simulation methods [7–18] rely on manually setting specific rules, leading to simulations that lack realism due to the inherent limitations of the rules themselves. In recent years, data-driven crowd simulation methods have shown some promise. These methods [19–22] usually build crowd simulations by predicting crowd behavior. However, while the crowd simulations developed through these methods demonstrate enhancements, there remains a pronounced disparity in the distribution of data characteristics when compared with real crowds. This discrepancy underscores the ongoing need to augment the realism of simulations.

Traditional crowd simulation methods are mainly based on rule-setting, and although they can effectively simulate pedestrian movement in various scenarios, they require cumbersome manual rule-making to match reality as much as possible. This process not only requires a lot of manual operation but also is almost impossible to complete the task of fully matching the characteristics of real groups. With the development of computer technology, obtaining real crowd data has become easier, and data-driven crowd simulation methods have also emerged. These methods rely on constructing crowd behavior based on real-world crowd trajectories and can generate more realistic crowd behavior compared to traditional methods [23]. Early data-driven approaches to crowd simulation involved the creation of crowd behavior

✉ Tianyu Huang
huangtianyu@bit.edu.cn

Dapeng Yan
yandapeng@bit.edu.cn

[1] Key Laboratory of Digital Performance and Simulation Technology, Beijing Institute of Technology, Zhong Guan Cun South Street, Beijing 100081, China

Published online: 29 April 2024

databases [24, 25] and the use of crowd patches [26]. These methods have a small calculation amount but are difficult to adapt to complex interactive scenarios, and the characteristics of real data are not fully utilized. With the development of artificial intelligence methods, researchers explore data-driven approaches based on deep learning to construct crowd simulations. These methods typically learn trajectory and social features from real human crowds and then use various techniques to build crowd simulations. For example, Li et al. [27] and Zhang et al. [28] combine deep learning techniques with the traditional ORCA [15] algorithm for crowd simulation. Zhang et al. [29] use neural networks to construct a model structure similar to the social force model [30]. These methods produce promising results in generating crowd simulations, particularly in collision avoidance between individuals, as they incorporate rules from traditional methods to enhance visual realism. However, due to the complexity of crowd behavior, they are limited to simulating crowd behaviors that fall within the scope of traditional algorithms. Other scholars [1, 31–33] employ reinforcement learning to construct crowd simulations, achieving commendable results. This approach enables agents to actively acquire interaction skills through value feedback, instead of learning real crowd interaction patterns from extensive datasets. Some scholars [19–22] construct pedestrian simulations or evacuation simulations by predicting the positions or velocities of the simulated crowd. After building a virtual group, these methods predict the future positions or behaviors of pedestrians through models. However, previous research [34] shows that pedestrian trajectories are multimodal and non-stationary, and predictions can only produce an average of various possibilities, which cannot fully capture the diversity of crowd movements. This averaging effect tends to make the simulated crowd behavior approximate the average behavior of complex crowds, which is more prone to collisions. Moreover, there are many cases in real crowds where people are too close to each other, and simulations based on this situation often result in more collisions, which reduces the visual realism of the simulation.

Addressing the aforementioned challenges, this paper furthers the study of deep learning-based methods for crowd simulation and proposes a model for generating natural pedestrian crowds. It aims to narrow the gap in data characteristics between simulated and real crowds to enhance the realism of the simulated groups and to address the issue of excessive collisions present in pure deep learning simulation models. This model employs the structure of Generative Adversarial Networks (GAN) [35], where the generator is used to create simulated crowd trajectories, and the discriminator is utilized to distinguish between real and generated data. Additionally, a collision classifier is introduced to identify collision avoidance behavior and reduce collisions in the generated data. When generating simulated crowds, the model guides each agent's actions based on the prior state of the pedestrian crowd, yielding crowd data that is independent of the scene and exhibits collision avoidance behavior. Throughout training model, it learns universal crowd features, unrelated to the scene, from crowd data across multiple disparate scenarios. Crowd features take into account both individual trajectories and social factors. In the extraction of trajectory features, the transformer [36] structure is applied directly after a straightforward data processing step. When extracting social features, a special data processing mechanism is proposed to transform the crowd's spatial position into a sequence and utilize the transformer structure for feature extraction. To enhance the collision-avoidance capability of the data generated by the model, an innovative learning mechanism is designed. This mechanism concurrently learns from real-world data and simulated data generated by traditional models. This dual learning approach ensures that the model not only captures the movement patterns of crowds in the real world but also effectively reduces crowd collisions. The overview of framework is shown in Fig. 1. The contributions of this paper can be summarized as follows:

- A virtual crowd generation model is created that generates the current velocity of each agent based on a set of agents' states and iteratively produces pedestrian crowd trajectories with collision avoidance behavior. The generated crowd behavior is not dependent on specific scenes and has generalizability, and the simulated crowd's data characteristic distribution approximates that of real crowds.
- A transformer module-based crowd feature extraction method is proposed. When the model learns crowd features, it combines the parallel computing mechanism of the transformer structure and the powerful sequence feature extraction ability to extract trajectory features of the crowd. For social feature extraction, a special data processing mechanism is designed to transform the spatial structure into a sequence after processing the group data, which is then extracted using the transformer structure.
- A novel model training mechanism is used to enhance crowd collision avoidance behavior. The model simultaneously learns from both real and the simulation data resolving collisions by traditional methods, enabling the generated simulated crowd not only to have realistic crowd behavior but also collision avoidance behavior with fewer collisions.

2 Related work

2.1 Crowd simulation method based on rules

The rule-based crowd simulation methods can be divided into macroscopic strategy models and microscopic models.

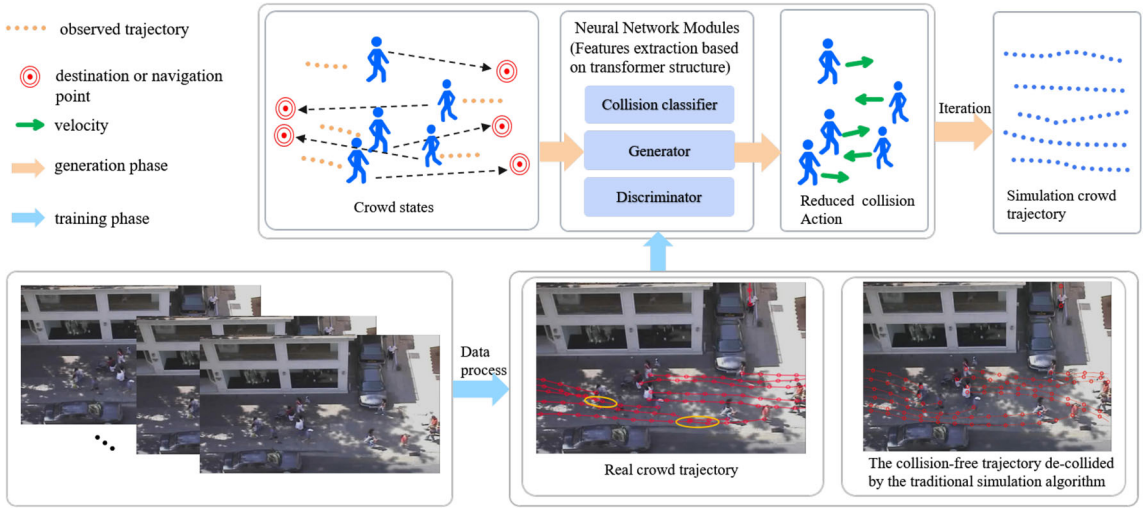


Fig. 1 Overview of our framework to generate the simulated crowd

Macroscopic strategy models rely on global rules and apply conditions to the entire group, where path planning and collision avoidance are controlled at the global level. Scholars have used different methods to construct macroscopic models, such as Colombo et al. [7] and Golas et al. [8] using the Continuum model, Narain et al. [9] using the Aggregate dynamics model, Lu et al. [10] and Tsai et al. [11] using Potential field to build crowd simulations. These methods can simulate the scheduling trends of the crowd well, but generally do not perform well in terms of interactions between individuals. Microscopic models focus on individual pedestrian movement, starting from a local perspective. Silva et al. [12] based their approach on visibility rules, Helbing et al. [37] on physical methods, Fiorini et al. [13] and Van Den Berg et al. [14–17] on agent velocity information, and Luo et al. [18] on agent's own attribute to simulate crowd movement and collision avoidance between groups. These methods require constant parameter adjustments to achieve good scheduling effects overall. As previously mentioned, rule-based methods often require repeated parameter adjustments and observation of group behavior at the macro and micro levels, which is time-consuming and dependent on experience. Some methods, such as [27–29], use deep learning to fit the parameters in rule-based methods or construct networks with similar structures to extend rule-based methods into data-driven methods. In this paper, we draw inspiration from the agent based approach to design our architecture, but do not directly reference the structure design of traditional methods or fit their corresponding parameters. Instead, we generate the current velocity of an agent based on its own state. Additionally, to improve the performance of the model, we propose a method that combines real-world crowd data and traditional method-generated data to learn crowd behavior.

2.2 Feature extraction for data-driven crowds

Crowd features refer to attributes extracted from crowd data, encompassing information on movement patterns, social interactions, and overall behavioral modes of the crowd. These characteristics typically exist in a high-dimensional form and are implicitly derived from complex crowd data through deep learning or machine learning methods. They are utilized in tasks that necessitate an understanding of the intrinsic laws and characteristics within the crowd. Zhao et al. [38, 39] classify crowd behavior using clustering to obtain group characteristics and apply them. Kim et al. [40] use statistical methods to calculate crowd motion patterns and kinematic characteristics. These methods reduce crowd characteristics to a few specific patterns, which cannot fully express complex crowd behavior. With the development of deep learning techniques, neural network algorithms have gradually been applied in crowd simulation. Some scholars [19, 20, 22] construct multi-layered perceptron (MLP) networks to extract crowd characteristics and simulate crowd behavior. However, this early neural network structure has a large computational cost and poor feature extraction capabilities. Yao et al. [21] propose a residual network to extract crowd characteristics, which considers both trajectory and social features of the crowd. However, their input requires pre-constructed crowd attributes, which may lose some effective features. Zhang et al. [29] construct a group feature extraction method for adjacent groups through residual networks. Amirian et al. [41] use Long Short Term Memory (LSTM) to extract individual trajectory information but do not consider social features among the crowd. Li et al. [27] extract crowd characteristics using LSTM and Gated Recurrent Unit (GRU) and combine them with the ORCA [15] algorithm to construct a crowd simulation.

Excellent progress has been made in exploring human motion patterns for deep learning-based crowd trajectory prediction tasks. However, due to the lack of micro collision avoidance strategies, it is difficult to apply these methods to crowd simulation. Nevertheless, these methods are worth learning from. In recent years, a large number of studies on crowd trajectory prediction have been based on temporal models. Some studies, such as [34, 42], use LSTM to extract crowd trajectory features, while others [43–45] use transformers for the same purpose. Many studies [34, 42, 44–46] have shown that crowd features should include both trajectory and social features. Additionally, recent studies in other tasks [47, 48] have demonstrated that integrating different types of data to enhance feature extraction and model performance resonates with approaches that combine various types of trajectories. These methods emphasize the importance of leveraging complex network architectures and data fusion techniques to capture intricate patterns in data-driven applications. These ideas are worth borrowing for crowd simulation tasks.

In summary, the characteristics of a crowd can be expressed as trajectory features and social interactions among individuals. In this paper, we use the transformer architecture to extract the trajectory features of a crowd. The transformer structure has the advantage of parallel computing, and its special attention mechanism can simultaneously consider global and local information. Moreover, we take into account the social attributes of the crowd and propose a method based on the transformer structure to extract the social features of a crowd.

2.3 Generative model in crowd task

Currently, generative models have been extensively studied and achieved good results in various fields, such as image tasks [49], video tasks [50], and text tasks [51], but have been less applied in crowd simulation tasks. Amirian et al. [41] first constructed a crowd simulation using GAN structure to generate individual trajectories, but did not consider the collision-avoiding interaction behavior among the crowd in the model structure. Li et al. [27] used a conditional variational autoencoder to fit the probability distribution of agent velocity.

In crowd trajectory prediction tasks, numerous studies have utilized GAN-based models [34, 44, 52–54]. These studies contend that crowd behavior is characterized by being multimodal, highly random, and uncertain, and that direct prediction can produce multiple possible average results more easily. GAN structures, on the other hand, are capable of generating multimodal results.

In this study, we draw inspiration from this idea and design an adversarial training-based model, proposing a generative framework that is suitable for generating more natural simulated pedestrian crowds. The model learns the conditional probability distribution of agent velocity given the current agent state and generates results that are independent of specific scenarios and have fewer collisions.

3 Method

In this chapter, we introduce the method of our model to generate simulated crowd trajectories based on the crowd state. Our model is a data-driven approach. Therefore, we train our model using real-world crowd data.

Like most methods [19–22, 27, 28, 40, 41], we define the environment as a 2D plane and the crowd as A with n individuals. Individual i in A is defined as agent i, with $i \in [1, n]$, and its trajectory is defined as $P_i = [p_i^1, p_i^2, ..., p_i^t]$, where t is the number of positions in the trajectory of agent i and $p_i^j = (x_i^j, y_i^j)$. $p_i^j \in \mathbb{R}^2$ represents the coordinate position of individual i at time j. The time interval between p_i^j and p_i^{j+1} is fixed as Δt. The velocity of agent i at time j is defined as v_i^j, $v_i^j \in \mathbb{R}^2$, $v_i^j = (p_i^{j+1} - p_i^j)/\Delta t$. The target direction of agent i at time j is defined as o_i^j, $o_i^j \in \mathbb{R}^2$, and o_i^j is a unit vector. The target direction $O_i = [o_i^1, o_i^2, ..., o_i^t]$ of agent i at all times. We define P_i^j as the position state of agent i at time j, $P_i^j = [p_i^1, p_i^2, ..., p_i^j]$. O_i^j is the target state of agent i at time j, $O_i^j = [o_i^1, o_i^2, ..., o_i^j]$. S_i^j is the state of agent i at time j, $S_i^j = [P_i^j, O_i^j]$.

Our method is an agent-based approach that predicts an agent's current velocity based on its past state and uses this to construct a crowd simulation. For agent i, we construct the following mappings:

$$\left[S_i^j, \bigcup_{i \neq o} S_o^j \right] \mapsto v_i^j \tag{1}$$

S_o^j represents the state of other agents that agent i can observe at time j.

For the group of agents, we construct a mapping from S^j to v^j, where S^j is the set of observable states of agents that can perceive each other's states, and v^j is the set of velocities that each agent should select based on the observed states of other agents. In other words, we can obtain the current velocities of all agents through a group of agents that can observe each other's states.

3.1 The structure of model

Our model is agent based, but to showcase more processing details, this section introduces the process of the model's handling of a group of agents. As shown in Fig. 2, in contrast

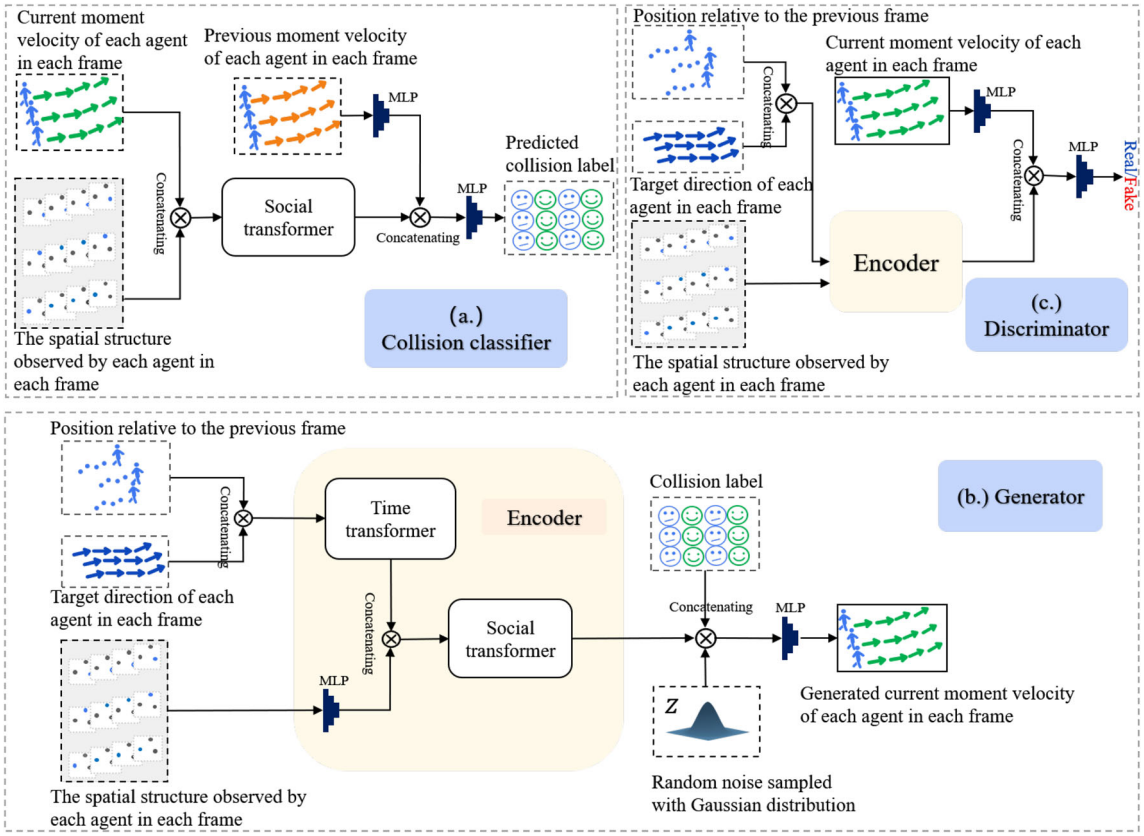


Fig. 2 Network structure of our model. It consists of three key components: **a** collision classifier. **b** generator, and **c** discriminator

with the standard GAN structure, our model consists of three parts: collision classifier, generator, and discriminator.

3.1.1 Collision classifier

The collision classifier is a binary classifier designed to predict if the velocity of agents at a specific moment will lead to a collision in the next moment, assuming the velocities of other agents remain constant. The classifier takes as inputs the spatial relationships between agents (i.e., the position information of all agents appearing in the same frame) with identity labels, and the velocities of each agent in the previous and current moments. The current velocities can be from real data or the generator's output. The output is the conditional probability that each agent's velocity will cause a collision in the next moment, based on the current moment's crowd position relationship and the previous moment's velocities of other agents.

The model structure of the collision classifier is shown in Fig. 2a. We concatenate the spatial relationship of agents at each time step with their corresponding velocity information from the previous time step to form a 4-dimensional vector. This is then aggregated by the social transformer module, which leverages the spatial characteristics of the crowd's movement at each time step to aggregate the social features of the group. We also map each agent's current velocity to a 32-dimensional vector using an MLP layer. Finally, we concatenate the social features and velocity information then feed them into an MLP layer with sigmoid as the final activation function for classification. The social transformer module requires input data that represents the spatial relationship of the crowd in a special data processing format, which we will describe in detail in a later section.

We assume that pedestrians' motion direction is influenced by their past trajectories and goals, and their movement decisions at each step are also influenced by their spatial relationships with other pedestrians. Based on this hypothesis, we design the internal structures of generator and discriminator.

3.1.2 Generator

The generator is used to learn the conditional probability distribution of agents' velocities. It takes as inputs the positional states and target states of a group of people who can observe each other's states, the spatial positional relationships between each agent and its observable agents at different moments, labels indicating whether the expected

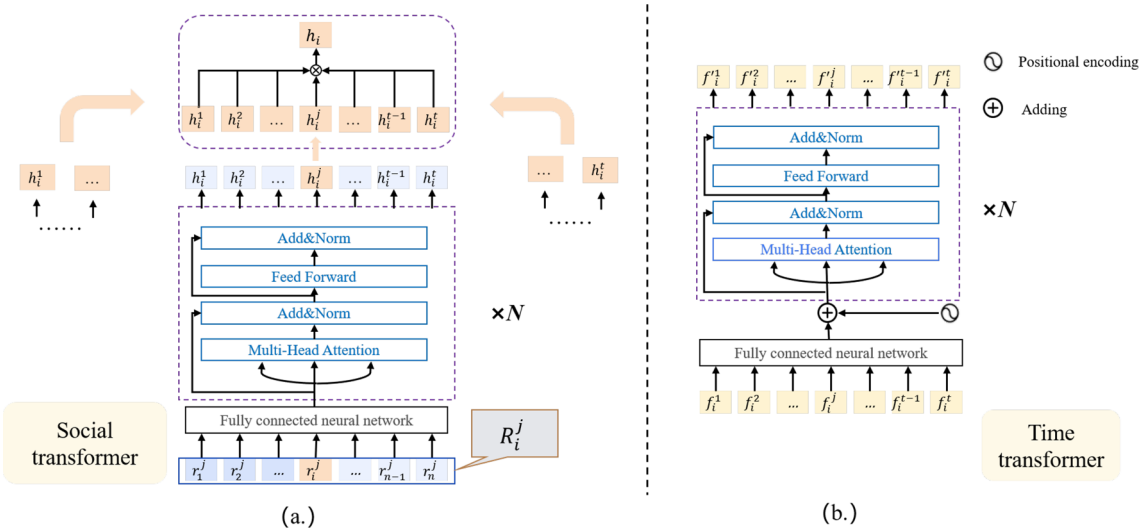


Fig. 3 Data processing mechanism of social transformer and time transformer

generated velocity of each agent will cause a collision, and random noise. Its output is the predicted velocity set $\hat{V}$ for each individual in the crowd, which is only influenced by the previous crowd features. The collision labels are used to train the model to generate fewer collision results. During training, the input data contain collision behavior, and velocities that will cause collisions are labeled as 1, while velocities that will not cause collisions are labeled as 0.

The generator structure is shown in Fig. 2b. The Encoder is used to extract the features of the agent group, which concatenates the trajectory information with the corresponding target direction at each time step to form a 4-dimensional vector. It then feeds into the time transformer module to extract temporal features. In the time transformer module, the 4-dimensional feature is finally processed into a 32-dimensional sequence feature. The trajectory information used here is the relative position of each agent at different time steps with respect to its previous time step, and the velocity information of the past time step is implicitly included in the trajectory information after processing by the time transformer. At the same time, we map the spatial structure of the crowd at each time step to 32-dimensional and then concatenate it with the trajectory features to feed into the social transformer module to extract the social features h, which combine the temporal and spatial features. Finally, we sample random noise from a standard normal distribution with a dimension of 16 and combine it with h and the collision label. Then an MLP layer is used to generate the velocity of each individual based on its trajectory and spatial relationships with other individuals in the past time step. The generated results will be discriminated by both the collision classifier and the discriminator. The transformer modules used in the social transformer and time transformer are both structured as transformer encoder blocks. However, their handling of input and output information is different. We will discuss this in detail in the next section.

3.1.3 Discriminator

The discriminator is used to determine the probability that the velocity information is real. It takes as input the crowd's position state, target state, and the spatial position relationship among agents at each time step, as well as the real velocity V or the generated velocity $\hat{V}$. Its output is the conditional probability that each agent's velocity at each moment is real.

The structure of the discriminator is shown in Fig. 2c, and one part of it has the same structure as the generator, which outputs the group feature h through the encoder. We map the velocity information to 32 dimensions through an MLP layer and then concatenate it with h. After that, we use an MLP layer with sigmoid as the final activation function to map the concatenated feature to a probability between 0 and 1.

3.2 Social and time transformer

Figure 3 illustrates two key transformer mechanisms in our model for processing data. These are used to extract social features and trajectory features from the data, respectively, which transform the spatial structure and trajectories of the crowd into latent feature vectors.

The social transformer is used to extract social features of the crowd during their movement. At the same moment, an agent can observe other agents that will have different impacts on its decision. We believe that the powerful attention mechanism of the transformer can effectively focus on such impacts. The model takes the different individual information in the scene at the same time as the input sequence of the transformer, that is, the length of the sequence of the

crowd movement is taken as the batch, and the number of agents is taken as the sequence length input to the social transformer. The transformer structure needs to use a certain characteristic as the identity label of the input sequence. Here we combine the spatial location information including identity with other information in the model to extract features. When setting the identity, we process the position relationship between agent i and the rest of agents at time j as a relative position relationship, where the coordinate position of agent i is (0,0) and the positions of other agents are the offset relative to agent i. We consider that the coordinate (0,0) of agent i has already characterized its identity information, and the position relationship with other individuals is independent of the sequence order. We map the spatial information of each agent to 32 dimensions via an MLP layer and then fuse it with other features (in the generator and discriminator, it is the trajectory sequence feature output by the time transformer, and in the collision classifier, it is the velocity feature) to obtain R_i^j, which serves as the input to the social transformer. As shown in the part (a) of Fig. 3, for the input information R_i^j of agent i at time j, it is first sent to a fully connected neural network (FC) to be mapped back to 32 dimensions and then sent to a transformer module. For the output of the transformer, at time j, we only keep the output h_i^j corresponding to agent i as the social feature of agent i at time j. Finally, we concatenate h_i^j with the social features of agent i at other times obtained by the same process to form the social feature h_i of agent i. For each individual in the crowd, we perform the above steps and then concatenate the social features of each agent, swap the dimensions of the feature sequence and social dimension, and adjust it to use the number of agents as the batch size and the length of the trajectory as the sequence length to obtain the output h of the entire social transformer as the social feature of the crowd.

The time transformer is shown in part (b) of Fig. 3. Its input is the hidden variable f that aggregates the trajectory and target features. We first send f to an FC layer to map it to a 32-dimensional vector and then add positional encoding to it. We then feed it into the transformer structure in the order of time steps. The positional encoding method used is the same as in [36], which helps the model fully utilize the sequential information of the features. When aggregating features, we use a mask mechanism to ensure that the velocity generated is only affected by the position and target information before the current time step.

3.3 Model optimization strategy

During the training process of our model, we optimize it not only using the GAN cost function L_{gan}, but also using the collision loss L_c and the L_2 loss between the real and generated velocities of the crowd.

The cost function is defined as follows:

$$L_{\text{gan}} = \min_G \max_D V(G, D)[\mathbb{E}_{x\sim p_{\text{data}}(x)}[\log D(x|C)] + \mathbb{E}_{z\sim p(z)}[\log(1 - D(G(z|C)))]]. \tag{2}$$

where C is the conditional information that needs to be input in the generator G and discriminator D.

The collision loss function is as follows:

$$L_c = -\frac{1}{n*t}\sum_{i=1}^{n}\sum_{j=1}^{t}(y_i^j log(\hat{y_i^j}) + (1 - y_i^j)\log(1 - \hat{y_i^j})). \tag{3}$$

The collision avoidance problem is crucial in crowd simulation, and we have designed a collision loss to reduce its occurrence. y_i^j is the true label indicating whether agent i's velocity at time j will cause a collision in the next time step, while $\hat{y_i^j}$ is the predicted label by the collision classifier. The collision loss is used to optimize both the collision classifier and the generator. For the collision classifier, only the gradient from real data is used during backpropagation to update the model. The collision classifier is trained in a self-supervised manner to help distinguish whether an agent's velocity will cause a collision, and its label value is calculated from the real data. For the generator, using L_c can teach it what collision behavior is and generate specific velocity information based on input labels. After the model is trained, inputting labels without collisions can make the model more likely to generate collision-free velocity information.

The formula of the L_2 loss function is as follows:

$$L_2 = \frac{1}{n*t}\sum_{i=1}^{n}\sum_{j=1}^{t}\min_k ||\hat{v_i^{j(k)}} - v_i^j||^2. \tag{4}$$

$\hat{v_i^j}$ represents the velocity generated by the generator for agent i at time j, and k denotes the number of times we generate. This approach is inspired by the diversity optimization strategy in [34], which encourages diversity in the generated results. L_2 is only used to optimize the generator, which approximates the generated velocities to the real velocities, helping the model to converge faster and produce more realistic results.

After the model training is completed and applied to simulation, we provide a set of initial positions and target directions for the individuals. By iteratively predicting the next time step velocity of each pedestrian and updating their positions, the target direction can guide the agent's path, and in complex paths, we can control the agent's movement by setting multiple waypoints as targets.

4 Experiments

We train our model using the ETH/UCY dataset. The ETH dataset [55] consists of pedestrian data from two scenes, ETH and Hotel. The UCY dataset [24] consists of pedestrian data from three scenes, Univ, Zara1, and Zara2. These datasets provide pedestrian trajectories sampled at 2.5Hz and contain rich social interactions among pedestrians.

4.1 Implementation details

When training the model, we divide the agents that coexist in the scene into groups, and the trajectory sequence length t of each group is the same. Before training, we randomly rotate the trajectories of each group to ensure that the data are independent of the scene. We determine that the coordinate distance between agents is less than 0.4m as a collision. During model training, collision labels are calculated based on this rule. We examine the collision situations of groups in the dataset and find that there are a large number of cases where the distance is too small. This is mainly due to two reasons: first, the crowd has clustering behavior; second, pedestrians tend to avoid collisions with the smallest cost, which means that pedestrians can avoid collisions with minimal sidesteps without changing their speed. This type of movement signal is very subtle, difficult to learn, and may cause the model to degrade [29]. To solve this problem, we propose a training method that combines traditional algorithms to generate trajectories with collision avoidance behavior. The traditional model can be any collision avoidance algorithm. During training, for the data that have experienced a collision, we process it with a social force model [30] to obtain a collision-free trajectory, which is then trained together with the original data. This will help the model learn how to avoid collisions.

The layers of all transformer modules in the model are set to 2, and the number of heads for multi-head attention is set to 4. The weight parameters of all linear layers are initialized using Kaiming initialization. The activation function for all MLP layers except the last one is set to ReLU, while the sigmoid function is used for the final discriminative layer. The learning rate is set to 0.0005.

We utilize a two-phase training strategy. In the first phase, the collision classifier is trained every two times, while the generator and discriminator are trained once. In the two training sessions of the collision classifier, one uses real crowd data and the other adds noise sampled from a 0.02 times standard normal distribution to the data. This mainly simulates the uncertainty of crowd behavior, which helps improve the generalization ability of the classifier. In the second phase, the parameters of collision classifier and discriminator are fixed, and generator is trained independently using collision-free labels. Additionally, the weights for the L2 loss are set to 0.1, aiming to achieve generated results with fewer collisions.

4.2 Performance analysis of social transformer

We propose social transformer for extracting social features of crowds based on the transformer module. As mentioned earlier, we adopt a unique data processing approach that sets the position of each agent at the same time to (0, 0) and calculates the relative positions of other agents to extract different social features for each agent, which are then fused into a holistic social feature. To verify the performance of this method, we compare it with the results of directly using transformer to extract features without special data processing. Additionally, we also use the max pooling approach employed in [34] to fit social features and compare the results.

To train and validate the model, the dataset is divided into an 8:2 ratio. The social feature extraction process involves training using the social transformer, transformer, and max pooling modules. Each method undergoes 2000 iterations.

As shown in Fig. 4, the method using the social transformer structure for fitting social features during training has a lower loss than the max pooling method and the approach that treats agents as a whole using the transformer. Additionally, the error on the validation set is significantly lower after training. Directly extracting features by treating the number of agents as a sequence results in slower convergence and less stable training due to the lack of agent identity information. This suggests that preserving individual agent identity information by calculating relative positions can produce better simulation results.

4.3 Real scene simulation experiment

4.3.1 Performance comparison

In our simulation experiments, we follow the basic settings of DeepORCA [27] and adopt the leave-one-out method to train the model, so the velocity probability distribution of the crowd in the simulation scenario is independent of the training data. We train the model for a total of 50 epochs, during which we validate the model's performance and save the models with good performance on all indicators. During generation, each agent is given its initial position and velocity (expressed in relative position) in the data and its destination (the last location where it appeared in the data). The target direction is calculated in real time based on the agent's current position and destination. Collision labels are set based on the real data. During generation, velocity information is generated through iterative looping, and each agent may have a different number of iterations, resulting in trajectories of different lengths.

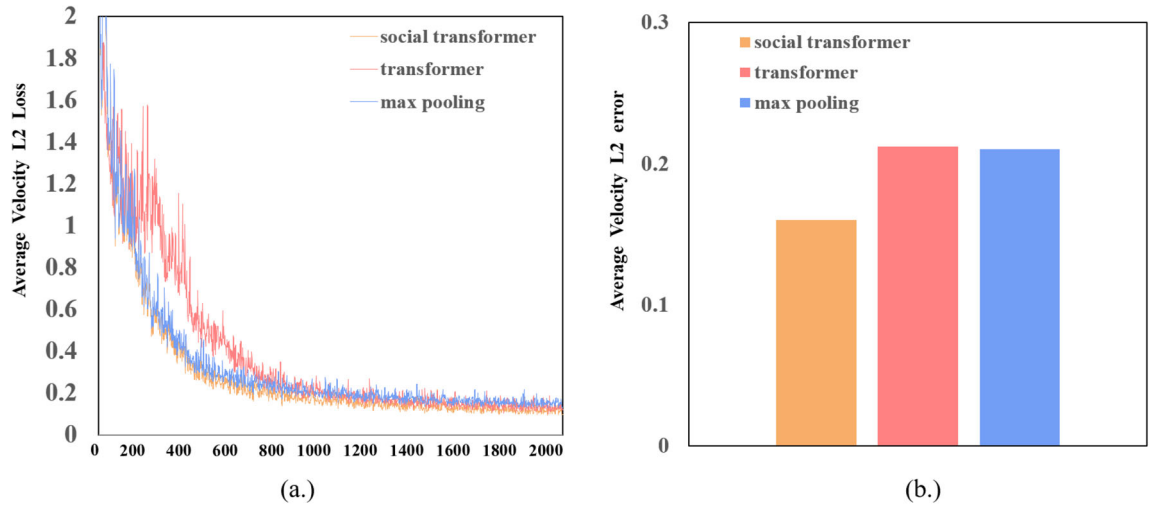


Fig. 4 Compare the performance of Social transformer, transformer, and max pooling. **a** The $L2$ errors between the velocities generated after each iteration in the training set and the actual velocities. **b** The $L2$ errors of velocities generated by each method in the validation set after 2000 iterations

To evaluate the performance of our method, we use the evaluation method from [38] to compare the simulation results with the real data. We define the trajectory length of agent i as t_i.

- Perpendicular deviation distance of agent i (m):

$$D_i = \sum_{j=1}^{t_i} PD(j)/t_i \tag{5}$$

$PD(j)$ is the perpendicular distance between p_i^j and the line connecting p_i^1 and $p_i^{t_i}$.

- Average speed of agent i (m/s):

$$V_i = \sum_{j=1}^{t_i} ||v_i^j||/t_i \tag{6}$$

- Average speed change of agent i (m/s^2):

$$\dot{V}_i = \frac{\sum_{j=1}^{t_i} \left(\left| \left\| v_i^j \right\| - \left\| v_i^{j-1} \right\| \right| \right) / \Delta t}{t_i} \tag{7}$$

- Average angel change of agent i (rad/s):

$$\dot{A}_i = \frac{\sum_{j=1}^{t_i} min(|\theta_i^j - \theta_i^{j-1}|, 2\pi - |\theta_i^j - \theta_i^{j-1}|)/\Delta t}{t_i} \tag{8}$$

θ_i^j is the angle at which agent i is moving at time j.

During the evaluation, we used Er_D,Er_V,$Er\_\dot{V}$, and $Er\_\dot{A}$, which represent the mean error rates between the generated and real results for each agent's average perpendicular distance, average speed, average speed change, and average angle change.

After simulating experiments with the data from various scenarios, we compare the results using the above metrics with various types of crowd simulation methods and summarize them in Table 1. SFM [30] and ORCA [15] are classic crowd simulation methods, and we adjust the parameters of the SFM method to make the speed as close as possible to the real crowd. For the results of ORCA, we use the data reported in DeepORCA [27]. Literature [22] is a method that uses deep learning to predict crowd behavior and build pedestrian simulations. Literature [41] is a method that generates single-agent trajectories based on GAN, and we refer to it as CrowdGAN based on its open-source code name. The trajectories generated by CrowdGAN are iteratively produced based on the first two positions of the agent. DeepORCA [27] is a method that uses deep learning to combine the ORCA algorithm to generate simulation trajectories, and its evaluation results are the best according to the metrics. The results we report come from their paper. The average perpendicular deviation distance in each metric can evaluate the overall shape of the generated trajectory, and speed, speed change, and angle change are used to comprehensively evaluate the speed of the generated results.

The results demonstrate that, apart from average speed, our approach produces results that are closer to the data distribution of real crowds than other methods, indicating that the crowds generated by our approach are more realistic. We fine-tune the parameters of the SFM algorithm to make it more similar to the average speed of real crowds, which leads to its best performance in the average speed metric. However, since the SFM algorithm needs to frequently change direction and speed to avoid collisions, it deviates from the

Table 1 Comparison of various indicators used to measure the closeness of the generated results to the real-world crowd data under different methods

Metric	Data set	SFM [30]	ORCA [15]	Literature [22]	CrowdGAN [41]	DeepORCA [27]	Our
Er_D	ETH	**0.25**	0.96	0.47	0.73	0.35	0.34
	Hotel	**0.20**	0.91	0.68	0.52	0.79	0.23
	Univ	0.75	0.96	0.59	0.38	0.40	**0.28**
	Zara1	0.66	0.95	0.56	0.35	0.20	**0.18**
	Zara2	0.67	0.46	0.33	0.23	0.12	**0.08**
Avg		0.51	0.85	0.53	0.44	0.37	**0.22**
Er_V	ETH	**0.11**	0.49	0.39	0.19	0.32	0.20
	Hotel	0.14	0.15	0.22	0.13	0.21	**0.07**
	Univ	**0.01**	0.42	0.28	0.06	0.18	0.16
	Zara1	0.03	0.05	0.17	**0.01**	0.10	0.05
	Zara2	0.05	0.18	0.11	**0.01**	**0.01**	0.02
Avg		**0.07**	0.26	0.23	0.08	0.16	0.10
$Er\_\dot{V}$	ETH	0.56	0.82	0.77	0.63	0.41	**0.24**
	Hotel	0.91	0.56	0.47	0.67	0.38	**0.23**
	Univ	1.19	0.19	0.54	0.20	**0.18**	0.38
	Zara1	0.82	0.35	0.51	0.54	**0.06**	0.29
	Zara2	0.65	0.45	0.33	**0.12**	0.44	0.28
Avg		0.83	0.47	0.52	0.43	0.29	**0.28**
$Er\_\dot{A}$	ETH	0.43	0.90	0.48	0.83	0.53	**0.21**
	Hotel	2.00	0.91	0.37	0.39	0.72	**0.22**
	Univ	7.09	0.50	0.90	0.62	**0.07**	0.60
	Zara1	4.97	0.45	0.76	0.38	0.32	**0.31**
	Zara2	4.70	0.54	0.71	0.57	**0.29**	0.37
Avg		3.84	0.66	0.64	0.56	0.39	**0.34**

Bold indicates better indicator results

behavior of real crowds in terms of speed and angle changes. CrowdGAN achieves good performance in the average speed metric mainly because it does not consider collision avoidance and focuses more on simulating the speed of crowds. In the evaluation results, our approach performs slightly worse than DeepORCA in terms of speed and direction changes in scenarios with more collisions, but our average results are still good across all datasets. This is because, to enhance the visual realism of crowd simulation, our approach slightly adjusts the agent's speed and direction to avoid collisions in scenarios with more collisions, while real crowds can avoid collisions by passing each other. However, in crowd simulation, the agent collision box is usually represented as a fixed-size circle, which will be considered as a collision in simulation.

4.3.2 Trajectories analysis

We present the trajectory and heatmap comparisons of different crowd simulation methods on the ETH dataset in Fig. 5. The number of simulated trajectories is the same as that in the dataset. The heatmap indicates the concentration of people in different areas of the scene. We compare the similarity between the simulated crowd heatmaps and the real crowd heatmap using Structural Similarity Index (SSIM). The results are summarized in Table 2. We did not reproduce the method of DeepORCA as it has not been open-sourced and the details of its parameters are not disclosed in the paper. Experimental result shows that our method can generate simulation trajectories similar to real-world scenarios for new crowd data, given the initial position, velocity, and destination of the crowd. Compared with our method, the trajectories generated by ORCA lack smoothness and the agent's position in the trajectory does not match the real state of the crowd. CrowdGAN method has limited generalization capability and produces trajectories with significant deviation when applied to new datasets. SFM and Literature [22] produce trajectories that are closer to the real crowd state, but in the heatmap, the positions of agents in our generated crowd are closer to the real crowd.

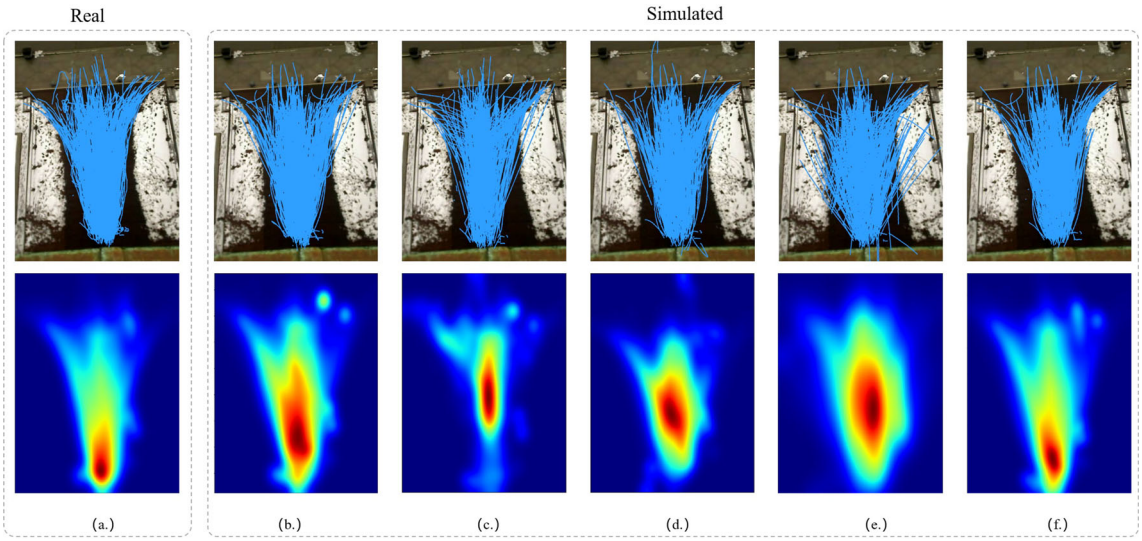


Fig. 5 Comparison of crowd trajectories and position heatmaps in ETH dataset. The upper part is the trajectories of real people or simulated people by various methods, and the lower part is the heat map composed of positions in the trajectories. **a** Real trajectories. **b** SFM [30]. **c** ORCA [15]. **d** Literature [22]. **e** CrowdGAN [41]. **f** Our method

Table 2 Comparison of structural similarity between simulated crowd heatmaps by different methods and a real crowd heatmap

-	SFM [30]	ORCA [15]	Literature [22]	CrowdGAN [41]	Our
SSIM	0.87	0.83	0.86	0.80	**0.92**

Bold indicates better indicator results

4.4 Artificial scene simulation experiment

Our model-generated results are applied to an artificial scenario to further verify the model's generalization and result diversity. In this experiment, a small continuous turning maze is constructed with navigation points set at the turning and exit points as the target directions for the agents to move. Three agents are placed at the entrance of the maze and are tasked with walking to the exit.

Figure 6 displays the simulation trajectories of various methods, including: (a) the simulation trajectory generated by the classical SFM method [30], which requires repeated attempts with different parameters to achieve satisfactory results and may exhibit unnatural repulsive behavior when agents are too close to each other; (b) the trajectory generated by the ORCA algorithm [15], which lacks the smooth characteristics of real crowds compared to our generated trajectories; (c) the result generated by [22], which fails to fully learn the characteristics of real crowds and lacks the smooth characteristics of real trajectories when agents change direction, and exhibits jitter when agents move without interaction; (d) the simulation trajectory generated by CrowdGAN, which cannot guide trajectory paths through navigation positions and can only construct segmented trajectories that are smoothed and connected as a whole after generation, and whose generated paths tend to overlap among different agents and require collision avoidance algorithms for simulation; (e) the multiple simulation trajectories generated by our method through the same model training. It can be seen that the agents can successfully walk from the entrance to the exit guided by navigation positions. The trajectories generated multiple times exhibit similar trends, but there are differences in details. Figure 7 shows more details, indicating that our method can produce multiple possible speed decisions based on agent states and has diverse results. Compared to other methods, the trajectories generated by our method are more in line with the actual crowd movement characteristics and have diversity.

4.5 Collision analysis

After providing the initial position and target direction of the crowds, the crowd movement processes are constructed using our model. All agents move to their opposite sides until they disappear from the scene. Figure 8 shows the processes of crowd movement. In all experimental groups, the crowds constructed by our model exhibited collision avoidance behavior.

In the same environments, we test ORCA [15], SFM [30], Literature [22], Social GAN [34], and the simulation results of our model trained only on real crowd data as comparative experiments. We evaluate the results by accumulating the collision times with a 2.5 Hz sampling rate of the crowd positions and summarize the results in Table 3. We adopt

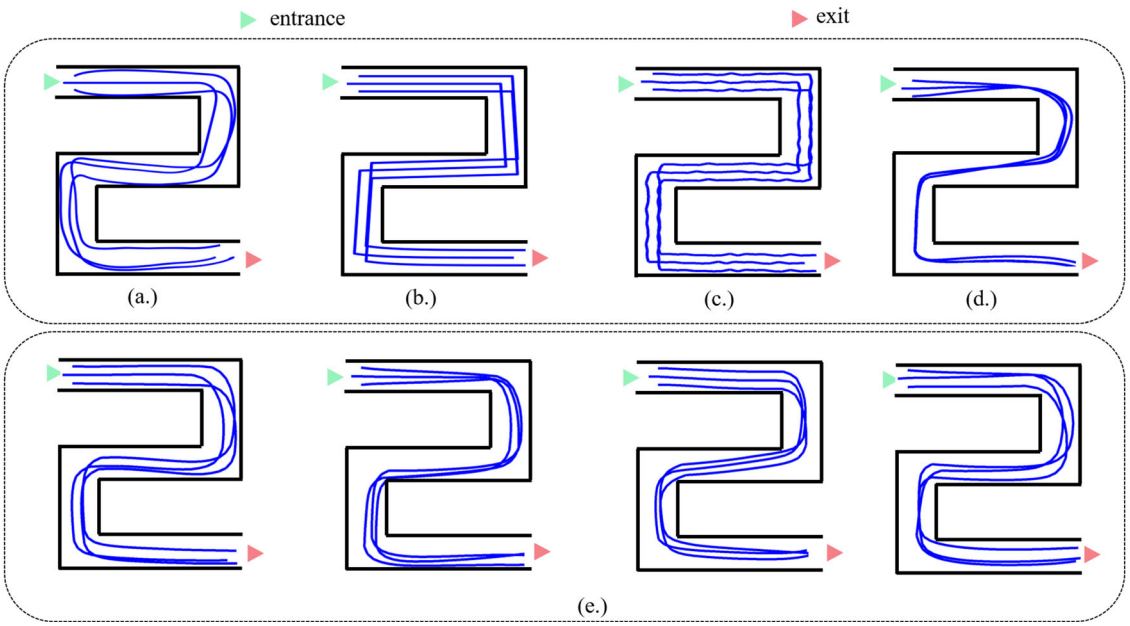


Fig. 6 Crowd simulation trajectories of different methods in artificial scene. **a** SFM [30]. **b** ORCA [15]. **c** Literature [22]. **d** CrowdGAN [41]. **e** Our method

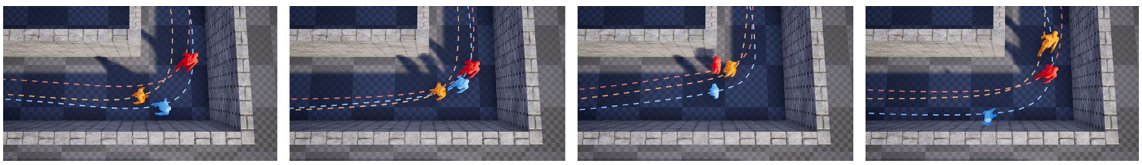


Fig. 7 Details of crowd trajectories generated multiple times by our model at the second turn in the artificial scene experiment

the average velocity of the crowd in the training set as the expected velocity for SFM and ORCA algorithms and adjust the parameters to produce relatively reasonable crowd simulation. For SFM algorithm, to reduce the unnatural repulsion caused by the repulsion force, the repulsion perception distance between agents is set to 0.3 m. For ORCA algorithm, to alleviate the problem that some agents have velocities close to zero for most of the time due to excessive avoidance considerations, we set the perception distance to 1.5 m. Social GAN is one of the state-of-the-art trajectory prediction methods that do not consider collision avoidance behavior. We generate its subsequent positions by predefining linear trajectories and iterating them. Our (Only) refers to the results obtained using our model trained only on the real dataset. Our method significantly lowers the collision rate in crowd simulations compared to trajectory prediction methods like Social GAN. Furthermore, our results outperform those mentioned in Literature [22] and other prediction-based approaches, demonstrating that our design reduces collisions in crowd simulations constructed with deep learning models. Our approach performs comparably to the ORCA algorithm in avoiding collisions in scenarios (a) and (b) and surpasses SFM in all scenarios. This indicates that our method can approach the avoidance capabilities of traditional methods in medium- to low-density crowd simulation scenarios. Our method performs worse than the ORCA algorithm in the dense crowd environment of scenario (c), mainly due to the uncertainty of real crowd behavior, and the fact that in dense states, the crowd tends to avoid collisions by squeezing spatial distances. This approach may produce smaller inter-agent distances, which are more likely to be judged as collisions. ORCA is more likely to find collision-free space and adjust the direction of movement accordingly. Furthermore, we compare the simulation results by training the model with only real data and the fusion of traditional method to remove collision in real data. The collision rate decreases when training the model with the fused data. This indicates that although the collision classifier can distinguish collision behavior when trained only with real data, the generator's ability to remove collision is not strong due to the limitation of the data itself. Traditional methods generate a large amount of collision-free behavior in simulated data, which helps to improve the performance of the generator when fusing this data for training.

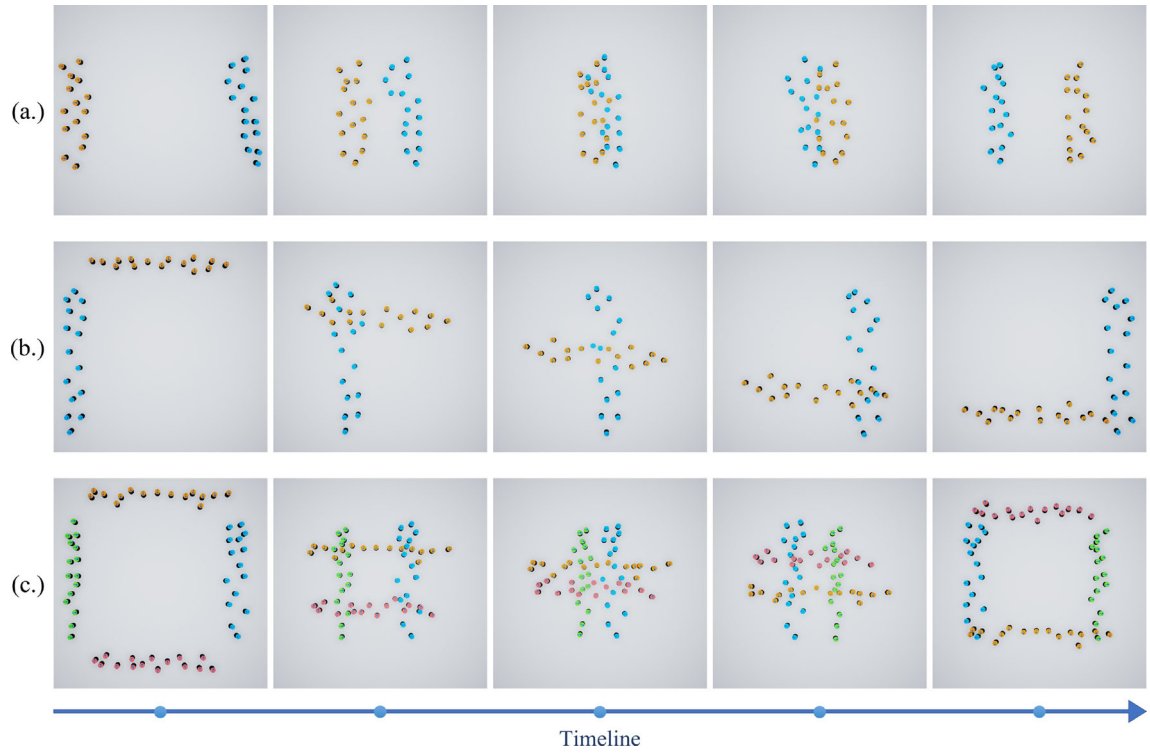


Fig. 8 Processes of crowd movement in collision avoidance experiments conducted by our model. **a** 15 agents on the left and right sides, moving to the opposite side, respectively. **b** 15 agents on the left side and the upper side, moving to the opposite side, respectively. **c** 15 agents in each of the four directions of up, down, left, and right, and they move to the opposite side, respectively

Table 3 Compare the number of collisions of each method in the collision experiment

-	SFM [30]	ORCA [15]	Literature [22]	Social GAN [34]	Our	Our (Only)
Scene a	6	**2**	16	16	**2**	8
Scene b	8	**3**	22	32	**3**	14
Scene c	67	**8**	75	96	31	53

Note:Bold indicates better indicator results

Table 4 Measure the effect of each loss function on model performance

L_{gan}	L_c	Avg Er_D	Avg Er_V	Avg $Er\_\dot{V}$	Avg $Er\_\dot{A}$	Avg Collision Times
×	×	0.22	**0.09**	**0.18**	**0.22**	46
✓	×	**0.19**	**0.09**	0.19	0.24	34
×	✓	0.21	0.10	0.31	0.37	21
✓	✓	0.22	0.10	0.28	0.34	**14**

Bold indicates better indicator results

4.6 Ablation experiment

Finally, we conduct ablation experiments to verify the effectiveness of the adversarial cost and collision loss. We first refer to the settings in real-scenario simulations and verify various average indicators with different loss modules. Then, we train the model using the entire dataset under different loss modules, test the collision frequency, and calculate the average results in each scenario according to the settings in the collision analysis experiment. The final results are summarized in Table 4.

The results demonstrate that incorporating the adversarial cost function L_{gan} and collision loss L_c has minimal impact on the overall trajectory trend and speed magnitude of the generated crowd. However, previous experiments have shown that the use of the generative model can yield diverse simulation outcomes. With regards to speed and direction changes, using L_c decreases model performance, suggesting

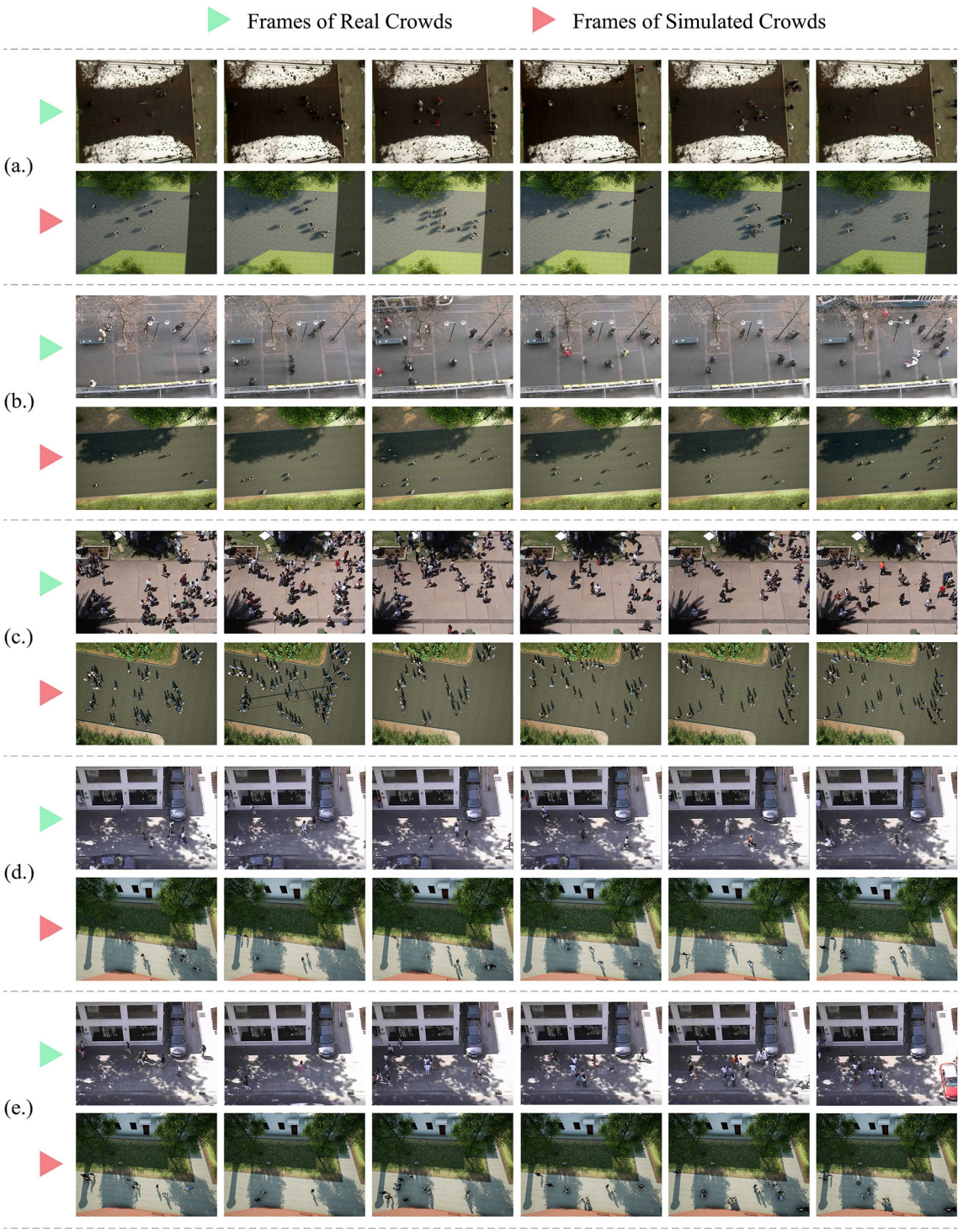


Fig. 9 Crowd simulations are constructed from the initial state of the crowd in each dataset. In each group, the upper part is the simulated scene and the lower part is the real scene. **a** ETH. **b** Hotel. **c** Univ. **d** Zara1. **e** Zara2

Fig. 10 Crowd simulation application scenario of the data generated by the model in the digital campus of Beijing Institute of Technology

that our approach avoids collisions by modifying the speed and direction of the agent. In terms of collision avoidance, it is observed that the inclusion of the L_c loss effectively reduces collisions, and the use of L_{gan} further reduces collisions. This is primarily attributed to the generative adversarial network producing diverse simulation outcomes, rather than converging to average behavior.

4.7 Simulation scene

Using the initial crowd distribution from each dataset, we construct the crowd simulation scene which exhibits similar pedestrian states to those in the dataset. The results are shown in Fig. 9.

We construct a digital campus of Beijing Institute of Technology using the Unreal Engine and generate crowd simulation using data generated by our model, further demonstrating the generalizability of our approach. The result is shown in Fig. 10.

5 Conclusion

We present a framework for constructing pedestrian simulations, capable of generating crowds exhibiting collision avoidance behavior. The generated results do not rely on specific environments and exhibit generalization. Compared to other methods, the simulated crowds generated by our framework are closer to real-world crowds. This work demonstrates the significant potential of generative methods in pedestrian simulation. Our framework incorporates the transformer architecture into data feature extraction for pedestrian simulation tasks and introduces a data processing method based on the transformer structure, showing excellent performance. During model training, we propose a mechanism that simultaneously trains the model with both real and simulated pedestrian data, which improves the model's performance.

Currently, our research only considers the interaction between pedestrians, without taking into account the interaction between pedestrians and the environment (such as buildings), which is a problem worth further study. In addition, we believe that generative model-based crowd simulation methods can be extended to virtual groups and interaction with real users. In future work, we will continue to investigate this.

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/s00371-024-03385-4.

Acknowledgements This research has been supported by the National Key Research and Development Program of China (No. 2020YFC2007200).

Author Contributions DY was responsible for the primary writing of the paper and coding the experiments. GD provided experimental equipment, financial support, and reviewed and revised the experimental section of the paper. KH handled the creation of the paper's images and built the code for comparative experiments. TH conducted a comprehensive review of the paper, revised the logical structure of the experimental part, and enhanced the quality and readability of the article.

Data availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest All authors have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

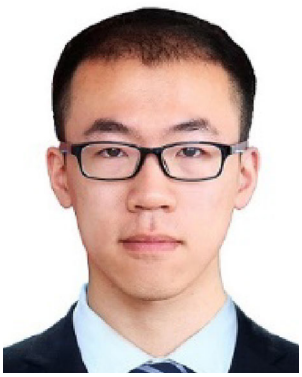
1. Lee, S.J., Popović, Z.: Learning behavior styles with inverse reinforcement learning. ACM Trans. Graph. (TOG) **29**(4), 1–7 (2010)
2. Curtis, S., Snape, J., Manocha, D.: Way portals: efficient multi-agent navigation with line-segment goals. In: Proceedings of the ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games, pp. 15–22 (2012)

3. Jordao, K., Charalambous, P., Christie, M., Pettré, J., Cani, M.-P.: Crowd art: density and flow based crowd motion design. In: Proceedings of the 8th ACM SIGGRAPH Conference on Motion in Games, pp. 167–176 (2015)
4. Kanyuk, P.: Virtual crowds in film and narrative media. In: Simulating Heterogeneous Crowds with Interactive Behaviors, vol. 217 (2016)
5. Ting, S.P., Zhou, S.: Snap: a time critical decision-making framework for MOUT simulations. Comput. Animat. Virtual Worlds **19**(3–4), 505–514 (2008)
6. Zhang, J., Jin, D., Li, Y.: Mirage: an efficient and extensible city simulation framework (systems paper). In: Proceedings of the 30th International Conference on Advances in Geographic Information Systems. SIGSPATIAL '22. Association for Computing Machinery, New York, NY, USA (2022)
7. Colombo, R.M., Rosini, M.: Existence of nonclassical solutions in a pedestrian flow model. Nonlinear Anal. Real World Appl. **10**(5), 2716–2728 (2009)
8. Golas, A., Narain, R., Lin, M.: A continuum model for simulating crowd turbulence. In: ACM SIGGRAPH 2014 Talks. SIGGRAPH '14. Association for Computing Machinery, New York, NY, USA (2014)
9. Narain, R., Golas, A., Curtis, S., Lin, M.C.: Aggregate dynamics for dense crowd simulation. In: ACM SIGGRAPH Asia 2009 Papers, pp. 1–8 (2009)
10. Lu, G., Chen, L., Luo, W.: Real-time crowd simulation integrating potential fields and agent method. ACM Trans. Model. Comput. Simul. **26**(4), 1–16 (2016)
11. Tsai, T.-Y., Wong, S.-K., Chou, Y.-H., Lin, G.-W.: Directing virtual crowds based on dynamic adjustment of navigation fields. Comput. Animat. Virtual Worlds **29**(1), 1765 (2018)
12. Silva, A.R.D., Lages, W.S., Chaimowicz, L.: Boids that see: using self-occlusion for simulating large groups on gpus. Comput. Entertain. **7**(4), 1–20 (2010)
13. Fiorini, P., Shiller, Z.: Motion planning in dynamic environments using velocity obstacles. Int. J Robot. Res. **17**(7), 760–772 (1998)
14. Berg, J., Lin, M., Manocha, D.: Reciprocal velocity obstacles for real-time multi-agent navigation. In: 2008 IEEE International Conference on Robotics and Automation, pp. 1928–1935 (2008)
15. Berg, J., Guy, S., Lin, M., Manocha, D.: Reciprocal n-Body Collision Avoidance, vol. 70, pp. 3–19 (2011)
16. Snape, J., Van Den Berg, J., Guy, S.J., Manocha, D.: Smooth and collision-free navigation for multiple robots under differential-drive constraints. In: 2010 IEEE/RSJ International Conference on Intelligent Robots and Systems. IEEE, pp. 4584–4589 (2010)
17. Snape, J., Van Den Berg, J., Guy, S.J., Manocha, D.: The hybrid reciprocal velocity obstacle. IEEE Trans. Robot. **27**(4), 696–706 (2011)
18. Luo, L., Chai, C., Ma, J., Zhou, S., Cai, W.: Proactivecrowd: Modelling proactive steering behaviours for agent-based crowd simulation. In: Computer Graphics Forum, vol. 37. Wiley Online Library, pp. 375–388 (2018)
19. Ma, Y., Lee, E., Yuen, R.: An artificial intelligence-based approach for simulating pedestrian movement. IEEE Trans. Intell. Transp. Syst. **17**(11), 3159–3170 (2016)
20. Wei, X., Lu, W., Zhu, L., Xing, W.: Learning motion rules from real data: neural network for crowd simulation. Neurocomputing **310**, 125–134 (2018)
21. Yao, Z., Zhang, G., Lu, D., Liu, H.: Learning crowd behavior from real data: a residual network method for crowd simulation. Neurocomputing **404**, 173–185 (2020)
22. Xiao, S., Han, D., Sun, J., Zhang, Z.: A data-driven neural network approach to simulate pedestrian movement. Phys. A Stat. Mech. Appl. **509**, 827–844 (2018)
23. Zhong, J., Li, D., Huang, Z., Lu, C., Cai, W.: Data-driven crowd modeling techniques: a survey. ACM Trans. Model. Comput. Simul. (TOMACS) **32**(1), 1–33 (2022)
24. Lerner, A., Chrysanthou, Y., Lischinski, D.: Crowds by example. In: Computer Graphics Forum, vol. 26. Wiley Online Library, pp. 655–664 (2007)
25. Charalambous, P., Chrysanthou, Y.: The pag crowd: A graph based approach for efficient data-driven crowd simulation. In: Computer Graphics Forum, vol. 33. Wiley Online Library, pp. 95–108 (2014)
26. Yersin, B., Maïm, J., Pettré, J., Thalmann, D.: Crowd patches: populating large-scale virtual environments for real-time applications. In: Proceedings of the 2009 Symposium on Interactive 3D Graphics and Games, pp. 207–214 (2009)
27. Li, Y., Mao, T., Meng, R., Yan, Q., Wang, Z.: DeepORCA: realistic crowd simulation for varying scenes. Comput. Animat. Virtual Worlds **33**(3/4), e2067 (2022)
28. Zhang, J., Li, C., Wang, C., He, G.: Orcanet: differentiable multi-parameter learning for crowd simulation. In: Computer Animation and Virtual Worlds, 2114 (2022)
29. Zhang, G., Yu, Z., Jin, D., Li, Y.: Physics-infused machine learning for crowd simulation. In: Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. KDD '22, pp. 2439–2449. Association for Computing Machinery, New York, NY, USA (2022)
30. Helbing, D., Molnar, P.: Social force model for pedestrian dynamics. Phys. Rev. E **51**(5), 4282 (1995)
31. Charalambous, P., Pettre, J., Vassiliades, V., Chrysanthou, Y., Pelechano, N.: GREIL-crowds: crowd simulation with deep reinforcement learning and examples. ACM Trans. Graph. **42**(4), 1–15 (2023)
32. Panayiotou, A., Kyriakou, T., Lemonari, M., Chrysanthou, Y., Charalambous, P.: CCP: Configurable crowd profiles. In: ACM SIGGRAPH 2022 Conference Proceedings. SIGGRAPH '22. Association for Computing Machinery, New York, NY, USA (2022)
33. Hu, K., Haworth, B., Berseth, G., Pavlovic, V., Faloutsos, P., Kapadia, M.: Heterogeneous crowd simulation using parametric reinforcement learning. IEEE Trans. Vis. Comput. Graph. **29**(4), 2036–2052 (2023)
34. Gupta, A., Johnson, J., Fei-Fei, L., Savarese, S., Alahi, A.: Social GAN: socially acceptable trajectories with generative adversarial networks. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 2255–2264 (2018)
35. Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., Bengio, Y.: Generative adversarial nets. In: Advances in Neural Information Processing Systems, vol. **27** (2014)
36. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. In: Advances in Neural Information Processing Systems, vol. 30 (2017)
37. Helbing, D., Buzna, L., Johansson, A., Werner, T.: Self-organized pedestrian crowd dynamics: experiments, simulations, and design solutions. Transp. Sci. **39**(1), 1–24 (2005)
38. Zhao, M., Cai, W., Turner, S.J.: Clust: simulating realistic crowd Behaviour by mining pattern from crowd videos. In: Computer Graphics Forum. Wiley Online Library, vol. 37, pp. 184–201 (2018)
39. Zhao, M., Turner, S.J., Cai, W.: A data-driven crowd simulation model based on clustering and classification. In: 2013 IEEE/ACM 17th International Symposium on Distributed Simulation and Real Time Applications. IEEE, pp. 125–134 (2013)
40. Kim, S., Bera, A., Best, A., Chabra, R., Manocha, D.: Interactive and adaptive data-driven crowd simulation. In: 2016 IEEE Virtual Reality (VR). IEEE, pp. 29–38 (2016)
41. Amirian, J., Van Toll, W., Hayet, J.-B., Pettré, J.: Data-driven crowd simulation with generative adversarial networks. In: Proceedings

of the 32nd International Conference on Computer Animation and Social Agents, pp. 7–10 (2019)
42. Alahi, A., Goel, K., Ramanathan, V., Robicquet, A., Fei-Fei, L., Savarese, S.: Social LSTM: Human trajectory prediction in crowded spaces. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 961–971 (2016)
43. Giuliari, F., Hasan, I., Cristani, M., Galasso, F.: Transformer networks for trajectory forecasting. In: 2020 25th International Conference on Pattern Recognition (ICPR). IEEE, pp. 10335–10342 (2021)
44. Lv, Z., Huang, X., Cao, W.: An improved GAN with transformers for pedestrian trajectory prediction models. Int. J. Intell. Syst. **37**(8), 4417–4436 (2022)
45. Yuan, Y., Weng, X., Ou, Y., Kitani, K.M.: Agentformer: Agent-aware transformers for socio-temporal multi-agent forecasting. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 9813–9823 (2021)
46. Mohamed, A., Zhu, D., Vu, W., Elhoseiny, M., Claudel, C.: Social-implicit: rethinking trajectory prediction evaluation and the effectiveness of implicit maximum likelihood estimation. In: Proceedings of the ECCV 2022: 17th European Conference on Computer Vision, Tel Aviv, Israel, October 23–27, 2022, Part XXII, pp. 463–479. Springer (2022)
47. Xie, Z., Zhang, W., Sheng, B., Li, P., Chen, C.L.P.: BaGFN: broad attentive graph fusion network for high-order feature interactions. IEEE Trans. Neural Netw. Learn. Syst. **34**(8), 4499–4513 (2023)
48. Sheng, B., Li, P., Ali, R., Chen, C.L.P.: Improving video temporal consistency via broad learning system. IEEE Trans. Cybern. **52**(7), 6662–6675 (2022)
49. Park, J., Kim, Y.: Styleformer: transformer based generative adversarial networks with style vector. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 8983–8992 (2022)
50. Yu, S., Tack, J., Mo, S., Kim, H., Kim, J., Ha, J.-W., Shin, J.: Generating videos with dynamics-aware implicit generative adversarial networks. arXiv:2202.10571 (2022)
51. Kojima, T., Iwasawa, Y., Matsuo, Y.: Making use of latent space in language GANs for generating diverse text without pre-training. In: Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Student Research Workshop, pp. 175–182. Association for Computational Linguistics, Online (2021)
52. Sadeghian, A., Kosaraju, V., Sadeghian, A., Hirose, N., Rezatofighi, H., Savarese, S.: Sophie: An attentive GAN for predicting paths compliant to social and physical constraints. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 1349–1358 (2019)
53. Amirian, J., Hayet, J.-B., Pettre, J.: Social ways: learning multi-modal distributions of pedestrian trajectories with GANs. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops (2019)
54. Huang, L., Zhuang, J., Cheng, X., Xu, R., Ma, H.: STI-GAN: multimodal pedestrian trajectory prediction using spatiotemporal interactions and a generative adversarial network. IEEE Access **9**, 50846–50856 (2021)
55. Pellegrini, S., Ess, A., Schindler, K., Van Gool, L.: You'll never walk alone: modeling social behavior for multi-target tracking. In: 2009 IEEE 12th International Conference on Computer Vision. IEEE, pp. 261–268 (2009)

Dapeng Yan received his M.S. degree in software engineering from Beijing Institute of Technology in 2019, currently working toward a Ph.D. in the School of Computer Science and Technology, Beijing Institute of Technology. His research interests include digital performance, computer simulation, deep learning, and crowd simulation.

Gangyi Ding received the B.E. degree from Peking University, China, in 1988, and Ph.D. at Beijing Institute of Technology, China, in 1993. He is a professor in the School of Computer Science and Technology at Beijing Institute of Technology. His research interests are computer simulation, software engineering, and digital performance.



Kexiang Huang received his B.E. degree in Computer Science from Beijing Institute of Technology in 2020 and is pursuing successive postgraduate and doctoral programs for a doctoral degree at Beijing Institute of Technology since September 2020. His research interests center on crowd simulation and digital performance.

Tianyu Huang received the B.E. degree in Computer Science from Jilin University, China, in 2002, and the Ph.D. degree in Computer Science from Beijing Institute of Technology, China, in 2007. She is currently an associate professor with the School of Computer Science and Technology, Beijing Institute of Technology. Her research interests include human motion modeling and simulation, crowds simulation, and virtual reality.